

CHAPITRE VI Les Algorithmes de Tri et de Recherche dans un tableau

I Introduction

Parmi les traitements les plus utilisés dans le domaine informatique, on distingue les traitements de tris qui sont utilisés presque tous les jours dans divers domaines.

- Tri des clients dans une banque
- Tri des articles dans un magasin
- Tri des élèves/étudiants dans une classe
- Tri des clubs dans un championnat

Les tris ne se limitent pas uniquement aux valeurs numériques. Ils peuvent être appliquées aux chaînes de caractères, aux lettres, aux dates, etc...

Il existe plusieurs méthodes de tri, nous allons nous limiter à l'étude des tris les plus célèbres et/ou les plus utilisés.

Définition

Un algorithme de tri est une suite finie d'instructions servant à mettre en ordre une séquence d'éléments selon un critère.

Les éléments à trier

II Les Algorithmes de tri

II.1 Le tri par sélection

Principe et étapes:

1. Commencer à l'emplacement $i = 1$
2. Parcourir le tableau à partir de i jusqu'à la fin et déterminer posmin = l'indice du plus petit élément de T
3. Si $T[\text{posmin}] \neq T[i]$, permuter $T[i]$ avec $T[\text{posmin}]$
4. Incrémenter i
5. Si on n'a pas atteint la fin du tableau, Aller à l'étape n°2

Procédure `tri_selection` (var T : TabEntier; n : entier)

Var

n, posmin, i, j : entier

Début

Pour i de 1 à $(n-1)$ Faire // 1. Commencer à l'emplacement $i = 1$

$\text{Posmin} \leftarrow i$

 Pour j de $i+1$ à n faire

 Si $(T[j] < T[\text{posmin}])$ alors

$\text{posmin} \leftarrow j$

 fini

 finpour

 si $(T[\text{posmin}] \neq T[i])$ Alors // 3. Si $T[\text{posmin}] \neq T[i]$, permuter $T[i]$ et $T[\text{posmin}]$
 permuter($T[\text{posmin}]$, $T[i]$)

 Finsi

Finpour // 4. Incrémenter i // 5. Si on n'a pas atteint la fin du tableau, Aller à n°2

Fin `tri_selection`

Procédure Permuter(var x, var y: entier)

Var

z : entier

Début

z $\leftarrow$ x

x $\leftarrow$ y

y $\leftarrow$ z

Fin permuter

II.2 Le Tri à bulles

Principe et étapes:

Le principe consiste à mettre en ordre deux par deux les éléments successifs de la séquence en effectuant une comparaison et une permutation si nécessaire. A la fin du premier parcours, le plus grand élément sera positionné en fin de séquence.

Le nouveau parcours d'arrêtera à l'avant dernière position cette fois.

1. Comparer les éléments d'indice 1 et 2, permuter pour les mettre en ordre
2. Comparer les éléments d'indice 2 et 3, permuter pour les mettre en ordre
3. Continuer ainsi de suite jusqu'à arriver à comparer les n-1 ième et le n ième élément
4. Si la taille de tableau est supérieure ou égale à 2, aller à l'étape 1 en diminuant la taille du tableau de 1

Procédure tri_bulles (var T : TabEntier ;n : entier)

Var

i : entier

Echange : booléen

Début

Répéter

Echange $\leftarrow$ faux

Pour i de 1 à n-1 Faire

Si T[i] > T[i+1] alors

Permuter(T[i], T[i+1])

Echange $\leftarrow$ vrai

Finsi

FinPour

n $\leftarrow$ n - 1

Jusqu'à (n=1) ou (NON échange)

Fin tri_bulles

II.3 Le Tri par insertion

Principe et étapes:

Le principe consiste à :

1. commencer à la position i=2
2. sauvegarder la valeur de l(ième élément dans une variable temporaire
3. effectuer un décalage vers la droite de tous les éléments qui sont supérieurs à l'ième élément
4. insérer l'ième élément à sa bonne position
5. incrémenter i
6. Si on n'a pas atteint la fin du tableau, Aller à l'étape n°2

Procédure tri_insertion (var T : TabEntier ; n : entier)

Var

i, j, elem : entier

Début

```

Pour i de 2 à n Faire // commencer à la position i=2
    elem ← T [ i ] // sauvegarder la valeur ième élément dans une variable temporaire
    j ← i - 1
    TantQue ( j > 0 ) ET ( T [ j ] > elem ) Faire // effectuer un décalage vers la droite
        T [ j+1 ] ← T [ j ] // de tous les éléments qui sont
        j ← j - 1 // supérieurs à l'ième élément
    FinTantQue
    T [ j ] ← elem // insérer l'ième élément à sa bonne position
FinPour // Si on n'a pas atteint la fin du tableau, Aller à l'étape n°2

```

Fin tri_insertion

III Les Algorithmes de recherche dans un tableau

III.1 La recherche séquentielle

La recherche d'un élément dans un tableau triée est un parcours conditionné : on s'arrête dès que l'élément est trouvé. La condition de sortie de la boucle doit aussi contenir le test sur la fin du tableau. On va utiliser une fonction qui retourne la position dans le tableau de l'élément s'il est trouvé, -1 sinon.

Version sans booléen	Avec booléen
Fonction recherche (T : TabEntier; n, X : entier) : entier Var i, pos: entier Début i ← 1 pos ← -1 TantQue (i ≤ n) ET (T[i] ≠ X) Faire i ← i+1 FinTantQue Si (T[i] = X) Alors pos ← i FinSi Retourne (pos) Fin recherche	Fonction recherche (T : TabEntier; n, X : entier) : entier Var i, pos: entier test : booléen Début i ← 1 pos ← -1 test ← faux TantQue (i ≤ n) ET (test = faux) Faire Si (T[i] = X) Alors pos ← i test ← vrai sinon i ← i+1 FinSi FinTantQue Retourne (pos) // ou retourne test Fin recherche

III.2 La recherche dichotomique

La recherche dichotomique ne peut être utilisée que lorsque le tableau est trié (dans un ordre croissant ou décroissant)

Le principe de la recherche dichotomique est le suivant :

1. Initialiser début à 1 et fin à n, la taille du tableau
 2. Calculer l'indice du milieu = (debut + fin) div 2
 3. Comparer l'élément recherché avec l'élément du milieu du tableau
 4. Si l'élément recherché est égal à l'élément du milieu du tableau, alors on arrête la recherche
 5. Si l'élément recherché est plus petit alors réduire fin à milieu – 1 et aller à l'étape n°2
 6. Si l'élément recherché est plus grand alors début $\leftarrow$ milieu+1, et aller à l'étape n°2
- La recherche s'arrête lorsque l'élément est trouvé ou lorsque début est plus grand que fin.

Fonction recherhdich (T : TabEntier; n, X : entier) : entier

Var

deb, fin, pos, mil : entier

Début

deb $\leftarrow$ 1

fin $\leftarrow$ n

pos $\leftarrow$ -1

répéter

mil $\leftarrow$ (d + f) div 2

si (x = T [mil]) alors

pos $\leftarrow$ mil

sinon

Si T[mil] < X alors

deb $\leftarrow$ mil+1

sinon

fin $\leftarrow$ mil -1

Finsi

Finsi

Jusqu'à (pos > 0) OU (deb> fin)

Retourne (pos)

Fin recherhdich

